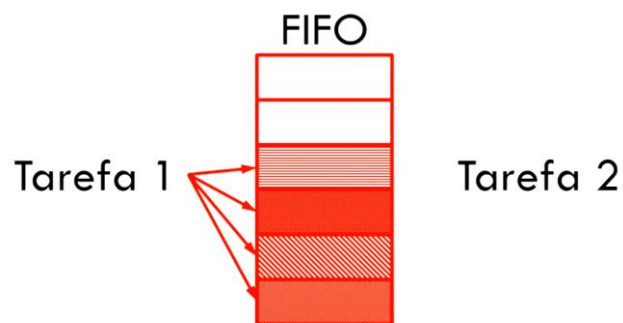


Experimento 2: Criando e Aplicando o Conceito de Tarefas com FreeRTOS

Valor: 2 pontos de 10

Pré-leitura

A FreeRTOS possui estruturas de comunicação entre tarefas do tipo Queues ou filas do tipo FIFO (First-in First-out).



Caso a Tarefa 1 alimente essa fila, a tarefa 2, que consome, pegaria a última tarefa da fila, que foi a primeira a entrar.

Atividade 1: Considere o PORTD do Arduino conectado a um conjunto de botões. A Tarefa 1 é responsável por mapear qual botão está pressionado e enviar para Tarefa 2 via ambiente compartilhado. A Tarefa 2 é responsável por acender um conjunto de LED's no PORTB com correspondência ao pino do PORTD acionado.

Obs: É necessária a inclusão da biblioteca queue que já vem com a FreeRTOS:

```
#include "queue.h"
```

Dispositivos necessários: Simulado ou Experimental

- Arduino ou Esp 32;
- Computador;
- botões;
- LED's
- Portoboard;

- Cabos de ligação;

Procedimentos

- Inclua as bibliotecas FreeRTOS e Queue
- Crie as tarefas_Ler_botoes e LED_acende
- Antes do Setup crie a instância para a Queue, exemplo (criando a fila):

```
QueueHandle_t xQueue_LED;
```

- No Setup adicione as tarefas a lista de aptos utilizando a xTaskCreate e avalie quais prioridades devem ser impostas a cada tarefa.
- Inicialize a fila dentro do setup utilizando a xQueueCreate:

```
xQueue_LED = xQueueCreate( 1, sizeof( byte ) );
```

Aqui temos 2 argumentos: o primeiro é o tamanho da fila, tamanho da estrutura de dados. (1 byte = 8bits = tamanho do PORT)

Funções para implementação das tarefas:

- Verificando se a fila existe:

```
if( xQueue_LED != 0 )
```

- Retorna quantidade de dados na fila:

```
uxQueueMessagesWaiting( xQueue_LED ) == 0
```

Com esta mesma função também é possível verificar se há dados na fila (se zero, está vazia).

- Escrevendo um dado na fila:

```
xQueueSend( xQueue_LED, &sLEDs, portMAX_DELAY);
```

Argumento 1 = fila;

Argumento 2 = endereço em que a informação está guardada (exemplo, variável byte que armazena os valores da porta);

Argumento 3 = tempo máximo que a tarefa pode esperar para armazenar algo na fila;

Neste exemplo, portMAX_DELAY significa uma espera eterna, quanto não vazia. Atentar que esta configuração pode criar um deadlock na tarefa.

- Ler a fila:

```
xQueueReceive( xQueue_LED, &rLEDs, ( TickType_t ) 10 )
```

Esta função tenta extrair dado da fila, caso consiga, retorna VERDADEIRO.

Podemos compará-la com pdPASS:

```
xQueueReceive( xQueue_LED, &rLEDs, ( TickType_t ) 10 ) == pdPASS
```

Assim, verificando esta condicional que indica a leitura, podemos dar prosseguimento a nossa execução.

Argumento 1 = fila;

Argumento 2 = região de memória que será armazenada a leitura;

Argumento 3 = tempo máximo que a função deve aguardar até que haja um dado na fila;

Tarefas:

Desenvolver um código de compartilhamento de dados com a queue;

Avaliar a possibilidade de deadlock com os tempos de espera escrita e leitura da fila.

Atividade 2:

Solucione a Atividade 1 fazendo uso do Mutex.

Procedimentos

A criação do Mutex na FreeRTOS está associada a estrutura do semáforo. Do ponto de vista da biblioteca, o Mutex é:

- Similar ao binário, entretanto, implementa a herança de prioridade. A herança de prioridade é uma implementação do FreeRTOS que aumenta a prioridade da tarefa que está usando o semáforo quando é interrompida por outra tarefa que também está tentando usar o semáforo. Isso garante que a tarefa que está usando o semáforo tenha a maior prioridade entre todas as outras que tentarem obtê-lo. Este método tenta minimizar o problema de inversão de prioridades.

As bibliotecas necessárias são:

```
#include <freertos/FreeRTOS.h>

#include <freertos/task.h>

#include <freertos/semphr.h>
```

Para criar uma mutex, é necessário declarar uma variável do tipo “**SemaphoreHandle_t**”. Esta usa o mesmo tipo de variável utilizada para semáforo, já que o semáforo e mutex compartilham as mesmas API's. Essa variável será utilizada como o identificador do mutex.

```
SemaphoreHandle_t xSemaphoreCreateMutex(void)
```

Para utilização do mutex temos 3 funções básicas:

- xSemaphoreCreateMutex(void)
- xSemaphoreTake
- xSemaphoreGive

- Criando a variável do mutex:

```
SemaphoreHandle_t myMutex;
```

- Criando o mutex dentro de uma tarefa:

```
void vATask(void * pvParameters){

    xSemaphore = xSemaphoreCreateMutex();

    if( xSemaphore != NULL ){

        Serial.println("mutex pronto para uso");

        return;

    }

    Serial.println("Nao pude criar o mutex");

}
```

- Para usar o mutex a x SemaphoreTake e Give servem para bloquear e desbloquear a seção critica:

```
void teste(char *str){  
    xSemaphoreTake(myMutex,portMAX_DELAY);  
    string_global = str;  
    serial.println(string_global);  
    xSemaphoreGive(myMutex);  
}
```

Tarefas:

- Resolver a atividade 1 com mutex
- Avaliar o tempo de espera para bloqueio evitando deadlock (atenção ao uso do portMAX_DELAY)
- Comparar as execuções em termos de possíveis deadlocks em relação ao método com queue